BEGINNER EDITION

25 Skills for Claude Code & Codex

Ek baar setup kar. Phir bas naam bol de. Copy, paste, done.

Ye version kiske liye hai: jisne abhi abhi Claude Code ya Codex install kiya hai aur samajh nahi aa raha ki aage kya. Har skill ke saath likha hai: ye kya karta hai, kab use karna hai, aur exactly kya bolna hai.

SHURU YAHAN SE

Pehle ye padh le (3 minute)

Agar tu ye pura page padh lega, baaki PDF apne aap samajh aa jayega. Skip mat karna.

Skill kya hai, ek line mein

Ek text file jo tere AI tool ko sikhati hai ki ek particular kaam **kaise** karna hai. Jaise tu naye intern ko ek baar samjha de, phir wo har baar waise hi kare.

Bina skill ke: har baar tu likhta hai "ye PDF padho, headings ke hisaab se notes banao, 5 bullets se zyada mat likho, formula mat simplify karo, jo nahi hai wo add mat karo..."

Skill ke saath: tu bolta hai `use pdf-to-notes`. Bas. Baaki instructions file mein pehle se hain.

KUCH WORDS JO AAGE AAYENGE

Terminal

Wo kaali screen jahan tu command type karta hai. Windows pe PowerShell, Mac pe Terminal. Claude Code yahin chalta hai.

Markdown (.md)

Plain text file jisme `#` se heading aur `-` se bullet banti hai. Notepad mein bhi khul jaati hai.

Folder / directory

Same cheez. `~` ka matlab tera home folder (C:\Users\tera-naam ya /Users/tera-naam).

Frontmatter

File ke sabse upar, do `---` ke beech ki 2 lines. Bas `name` aur `description`. Yahi tool padhta hai.

Claude Code

Anthropic ka terminal wala AI tool. Skills `~/.claude/skills/` folder se uthata hai.

Codex

OpenAI ka terminal wala AI tool. `AGENTS.md` file padhta hai, aur naye versions mein `~/.codex/skills/` bhi.

Ek honest baat

Agar tune kabhi terminal nahi khola, pehle 20 minute YouTube pe "terminal basics" dekh le. `cd`, `mkdir`, `ls` itna kaafi hai. Uske baad ye PDF 10x kaam ka hoga.

HAATH SE KARKE DEKH

Pehli skill, 5 minute mein

Claude Code ke liye. Codex wale agle page pe jaayein. Ek baar ye kar liya, baaki 24 same process hain.

1 **Terminal khol aur folder bana**

Ye command copy kar ke paste kar de. Ye ek folder banata hai jahan skill rahegi.

```
mkdir -p ~/.claude/skills/pdf-to-notes
```

2 **Us folder mein SKILL.md naam ki file bana**

Koi bhi text editor (VS Code, Notepad, kuch bhi). File ka naam exactly `SKILL.md`, capital letters mein. Path ye hoga:

```
~/.claude/skills/pdf-to-notes/SKILL.md
```

3 **Skill #1 ka pura block copy kar ke file mein paste kar**

Padhai section mein sabse pehli skill hai. `---` se shuru, aakhri rule tak. Save kar de.

4 **Claude Code band kar ke dobara khol**

Naya skill tabhi dikhta hai jab tool restart ho. Terminal mein `claude` likh ke enter.

5 **Bol ke dekh**

```
use the pdf-to-notes skill on lecture4.pdf
```

Agar Claude notes banana shuru kare, ho gaya. Tune apni pehli skill install kar li.

Check: folder ka naam aur frontmatter ka name same hai?

Folder `pdf-to-notes`, frontmatter mein `name: pdf-to-notes`. Alag hua to skill nahi uthegi. Ye sabse common galti hai.

Personal vs project skill

`~/.claude/skills/` mein daali to har project mein milegi. Sirf ek project ke liye chahiye to us project ke andar `.claude/skills/` banao (bina `~` ke). Shuru mein personal wala hi use kar.

CODEX WALE

Codex mein kaise lagaye

Codex skills ko thoda alag tarike se padhta hai. Do raaste hain:

1 **Raasta A (sabse pakka): AGENTS.md**

Apne project ke root folder mein `AGENTS.md` file bana (ya pehle se hai to usi mein). Skill ki **body**
(frontmatter ke neeche wala hissa) copy kar ke usme paste kar de. Codex ye file har baar padhta hai.

```
# project root mein, ek hi command se append karne ke liye
cat skill-body.md >> AGENTS.md
```

2 **Raasta B: ~/.codex/skills/ folder**

Codex ke naye versions mein ye folder bhi support hai, bilkul Claude Code jaisa structure. Apne installed version ke docs mein ek baar confirm kar lena.

Dono ke liye ek hi baat

Skill ki body plain markdown hai. Wo portable hai. Claude Code se Codex, Codex se kuch aur. Sirf wrapper alag hai, kaam wahi.

AGAR KAAM NA KARE

Troubleshooting, isi order mein

Dikkat	Check kar
Skill uth hi nahi rahi	Description padh. Usme "Use when..." wala hissa hai? 99% baar yahi problem hai.
"Skill not found"	Folder ka naam aur `name:` same hai? File ka naam exactly `SKILL.md` hai?
Naya skill dikh nahi raha	Tool restart kiya? Bina restart ke nahi dikhta.
Galat folder	`ls ~/.claude/skills/` chala ke dekh, skill wahan hai?
Sab sahi hai, phir bhi nahi	Tools tezi se badalte hain. Apne version ke docs mein "skills" search kar. Folder paths sabse pehle badalte hain.

SAARI 25 EK NAZAR MEIN

Index

Sab ek saath mat daal. Apni category se 2-3 uthao, ek hafta chalao, phir aur add karo.

Padhai	1. pdf-to-notes · 2. paper-explainer · 3. flashcard-maker · 4. doubt-solver
Job aur career	5. resume-tailor · 6. cover-letter · 7. interview-prep · 8. linkedin-post
Coding	9. commit-writer · 10. test-writer · 11. code-explainer · 12. bug-repro · 13. pr-reviewer · 14. refactor-planner
Data aur office	15. sheet-cleaner · 16. invoice-parser · 17. meeting-notes · 18. email-triage
Content banane wale	19. reel-scripter · 20. thumbnail-brief · 21. caption-writer · 22. content-repurposer
Roz ka kaam	23. weekly-review · 24. decision-log · 25. trip-planner

Har skill ke page pe 4 cheezein milengi

- **Ye kya karta hai:** ek line, plain bhasha mein
- **Kab use kare:** kaunse moment pe ye kaam aayega
- **Bol ke dekh:** exactly kya type karna hai terminal mein
- **Copy-paste block:** pura SKILL.md, jaisa hai waisa daal do

SKILLS 1-4

Padhai

Students ke liye. Notes, papers, revision, doubts.

01 PADHAI
pdf-to-notes

YE KYA KARTA HAI

Koi bhi PDF de, ye usse saaf notes bana deta hai. Jaise koi topper dost tere liye notes bana ke de.

KAB USE KARE

Exam se pehle. Ya jab 40 page ka chapter padhne ka man na ho.

BOL KE DEKH

`use pdf-to-notes on chapter3.pdf`

Copy this into `~/.claude/skills/pdf-to-notes/SKILL.md`

```
---
name: pdf-to-notes
description: Turn a PDF into clean, structured, revision-ready notes. Use when the user
shares a PDF, research paper, textbook chapter, or lecture handout and wants notes, a
summary, or "isko samjha do".
---

# PDF to Notes

Jab user PDF de:

1. Pehle pura padho. Jab tak poora na padh lo, likhna shuru mat karo.
2. Document ka apna structure follow karo. Apna naya structure mat thopo.
3. Har section ke liye likho:
   - **Ek line ka matlab** (plain language mein)
   - **Key points** (3-5 bullets, har bullet ek complete soch)
   - **Terms** (naye words + unka simple matlab)
4. Ant mein ek "Ye yaad rakhna" box: sirf wo 5 cheezein jo exam ya kaam mein sach mein
chahiye.

Rules:
- Jo PDF mein nahi likha, wo mat add karo. Gap ho to bolo "PDF mein nahi hai".
- Formula jaisa hai waisa likho, simplify mat karo.
- Agar PDF mein diagram hai jo padha nahi ja sakta, bol do. Chup mat raho.
```

02 PADHAI

paper-explainer

YE KYA KARTA HAI

Research paper ko teen layers mein samjhata hai: pehle ek para, phir asli baat, phir honest review.

KAB USE KARE

Jab arXiv link ya paper PDF mile aur sar ke upar se jaa raha ho.

BOL KE DEKH

`use paper-explainer, here is the link: arxiv.org/abs/...`

Copy this into `~/.claude/skills/paper-explainer/SKILL.md`

```
---
name: paper-explainer
description: Explain a research paper in plain language, layer by layer. Use when the
user shares an arXiv link, a paper PDF, or asks "this paper kya keh raha hai".
---

# Paper Explainer

Teen layers mein samjhao, isi order mein:

**Layer 1: Ek paragraph.** Ye paper kya keh raha hai, jaise dost ko bata rahe ho. Koi
jargon nahi.

**Layer 2: Asli baat.**
- Problem kya thi, aur pehle log kaise solve karte the
- Inhone alag kya kiya (yahi paper ka core hai)
- Kaise prove kiya ki ye better hai
- Numbers jo matter karte hain

**Layer 3: Honest review.**
- Ye kis case mein kaam nahi karega
- Authors ne khud kya limitation likhi hai
- Kya koi aur ne isko reproduce kiya

Rules:
- Har technical term pehli baar aaye to turant define karo.
- Result overclaim mat karo. Paper jitna keh raha hai utna hi bolo.
- Agar paper kamzor hai, bol do. Har paper achha nahi hota.
```

03 PADHAI
flashcard-maker

YE KYA KARTA HAI

Notes ya chapter se chhote question-answer cards banata hai. Ek card, ek fact.

KAB USE KARE

Exam se 3-4 din pehle, jab revise karna ho.

BOL KE DEKH

`use flashcard-maker on my notes.md, Anki format`

Copy this into `~/.claude/skills/flashcard-maker/SKILL.md`

````
---
name: flashcard-maker
description: Turn notes or a chapter into spaced-repetition flashcards in question-answer
format. Use when the user wants flashcards, Anki cards, or revision questions from study
material.
---

# Flashcard Maker

Har card ka format:
```
Q: <ek single question>
A: <chhota jawab, max 2 lines>
```

Rules jo card ko accha banate hain:
1. **Ek card = ek fact.** Do cheezein ek card mein mat daalo.
2. **Question specific ho.** "Photosynthesis kya hai?" bekaar hai. "Photosynthesis ke
light reaction mein oxygen kahan se aati hai?" achha hai.
3. **Answer chhota ho.** Agar answer paragraph ban raha hai, question tod do.
4. **Definition ke saath ek example bhi.** Alag card mein.
5. Jo cheez samajhne wali hai uske liye "kyun" wala card banao, "kya" wala nahi.

Ant mein batao kitne cards bane aur kaunse topics cover hue.
Agar user bole "Anki format", to tab-separated do taaki direct import ho jaye.
````

04 PADHAI
doubt-solver

YE KYA KARTA HAI

Problem ko step by step solve karwata hai, seedha answer nahi deta. Tu seekhta hai, copy nahi karta.

KAB USE KARE

Homework ya practice problem mein atak gaya ho.

BOL KE DEKH

`use doubt-solver: integrate x squared times e to the x`

Copy this into `~/.claude/skills/doubt-solver/SKILL.md`

```
---
name: doubt-solver
description: Walk through a problem step by step so the user actually learns it, instead
of handing over the final answer. Use for homework, practice problems, maths, physics, or
any "ye solve kar do" request.
---

# Doubt Solver

**Sabse pehle: seedha answer mat do.** Agar answer de diya, user ne kuch nahi seekha.

Process:
1. **Problem restate karo** apne words mein. Confirm karo ki sahi samjhe.
2. **Poocho:** "tu kahan tak pahuncha?" Agar user ne try kiya hai, wahin se shuru karo.
3. **Pehla step do, aur ruk jao.** Poocho: "ab next kya lagta hai?"
4. Agar user galat bole, bolo galat hai, par batao ki **soch kahan patri se utri**. Sirf
"no" mat bolo.
5. Jab user pahunch jaye, **poora solution ek baar clean likho** taaki wo revise kar
sake.
6. Ant mein: "isi type ka agla problem aaye to ye pattern dhundhna".

Exception: agar user saaf bole "bas answer chahiye, samay nahi hai", de do, par neeche ek
line mein method zaroor likho.
```

SKILLS 5-8

Job aur career

Apply karne se lekar interview tak.

05 JOB AUR CAREER
resume-tailor

YE KYA KARTA HAI

Tera resume aur job post dono leta hai, phir resume ko us job ke hisaab se reorder aur rewrite karta hai. Jhooth kabhi nahi.

KAB USE KARE

Har baar apply karne se pehle. Haan, har baar.

BOL KE DEKH

`use resume-tailor with resume.pdf and this job post: (paste)`

Copy this into `~/.claude/skills/resume-tailor/SKILL.md`

```
---
name: resume-tailor
description: Rewrite and reorder a resume to match a specific job posting, without
inventing experience. Use when the user shares a resume plus a job description and wants
to apply.
---

# Resume Tailor

Chahiye: resume + job post. Dono mein se ek bhi missing ho to maang lo.

Process:
1. **Job post se nikalo:** required skills, nice-to-have, aur wo exact words jo posting
baar-baar use karti hai.
2. **Resume se match karo.** Har requirement ke saamne likho: strong match / partial /
missing.
3. **Reorder karo.** Jo cheez is job ke liye sabse relevant hai wo upar. Baaki neeche ya
hata do.
4. **Bullets dobara likho** taaki posting ki language se match karein, par **sirf wo
cheezein jo user ne sach mein ki hain.**
5. **Missing wali list alag do:** "ye tere resume mein nahi hai, agar kiya hai to add
kar".

Bullet ka format: `<kya kiya> | <kaise> | <result, number ke saath>`

**Kabhi bhi jhooth mat likho.** Koi skill, degree, ya job add mat karo jo user ne nahi
batayi. Interview mein wahi phasata hai.
```

06

JOB AUR CAREER

cover-letter

YE KYA KARTA HAI

200-250 word ki cover letter jo template jaisi na lage. Recruiter ko cold email bhi isi se likhwa.

KAB USE KARE

Jab application mein letter maanga ho, ya recruiter ko pehli baar mail karna ho.

BOL KE DEKH

`use cover-letter for this role at Razorpay: (paste job post)`

Copy this into `~/.claude/skills/cover-letter/SKILL.md`

```
---
name: cover-letter
description: Write a short, specific cover letter from a resume and a job post. Use when
the user is applying for a job or internship and needs a letter or a cold email to a
recruiter.
---

# Cover Letter

Length: **200 se 250 words. Bas.** Lambi letter koi nahi padhta.

Structure:
1. **Pehli line:** kaunsi role, aur ek specific reason ki *is* company mein kyun. Company
ke baare mein kuch asli baat likho, "your reputation for excellence" type ghisa-pita
nahi.
2. **Beech ka hissa:** ek ya do kaam jo user ne kiye aur jo is job se seedha judte hain.
Poora resume dobara mat likho.
3. **Aakhri line:** simple, confident. "Happy to walk you through X."

Rules:
- Resume ke bullets copy-paste mat karo. Unke peeche ki kahani likho.
- "I am writing to express my interest": kabhi nahi.
- "passionate", "dynamic", "synergy": kabhi nahi.
- Aisi normal language jo user khud bol sake.
- Company ka naam sahi spell karo. Ek baar aur check karo.
```

07

JOB AUR CAREER

interview-prep

YE KYA KARTA HAI

Role ke hisaab se 12-15 likely questions, har ek ka draft answer tere apne experience se, aur 3 sawaal jo tu unse poochhega.

KAB USE KARE

Interview se 2-3 din pehle.

BOL KE DEKH

`use interview-prep, interview is for SDE intern at Zomato, resume attached`

Copy this into `~/.claude/skills/interview-prep/SKILL.md`

```
---
name: interview-prep
description: Generate likely interview questions for a specific role and help the user
build answers from their real experience. Use when the user has an interview coming up.
---

# Interview Prep

Chahiye: job post + user ka resume ya experience.

Do:
1. **12 se 15 questions** banao, teen category mein:
   - Technical (role ke hisaab se)
   - Behavioural ("ek baar jab tune...")
   - Company/role specific ("ye job kyun")
2. Har question ke liye:
   - **Interviewer asli mein kya dekh raha hai** (question ke peeche ka maqsad)
   - **Ek draft answer, user ke apne experience se.** Banaya hua nahi.
   - **Aage kya poochenge** (follow-up)
3. Behavioural ke liye STAR use karo: Situation, Task, Action, Result. Result mein number
ho to sabse achha.
4. Ant mein: **3 sawaal jo user interviewer se poochhe.** Ye log bhool jaate hain aur
yahi impression banata hai.

Rule: agar user ke background mein kisi question ka jawab nahi hai, bol do: "iska answer
tere paas abhi nahi hai, ye soch ke rakh le". Jhootha example mat banao.
```

08

JOB AUR CAREER

linkedin-post

YE KYA KARTA HAI

Jo kaam tune kiya, usse 150-200 word ki LinkedIn post. Cringe wali nahi, insaan jaisi.

KAB USE KARE

Kuch ship kiya, kuch seekha, kuch mila.

BOL KE DEKH

`use linkedin-post: I built a Telegram bot that tracks my expenses, took 2 weekends`

Copy this into `~/.claude/skills/linkedin-post/SKILL.md`

```
---
name: linkedin-post
description: Turn something the user did into a LinkedIn post that sounds human. Use when
the user shipped a project, learned something, got a job, or wants to post about their
work.
---

# LinkedIn Post

Length: 150-200 words. Pehli 2 lines sabse important. Baaki "see more" ke peeche chhup
jaata hai.

Structure:
1. **Hook:** ek specific line. Number ya ek asli detail. Question mat pucho.
2. **Kya hua:** asli kahani, chhoti.
3. **Kya seekha:** ek cheez, jo doosre bhi use kar sakein.
4. **End:** simple. Agar CTA hai to ek, warna kuch nahi.

Banned (ye LinkedIn ko cringe banate hain):
- "I'm humbled and honoured to announce"
- "Thoughts?" bas aise hi chipka dena
- Har line ke baad ek line ka gap, poori post bhar
- Emoji bullet points
- Fake vulnerability ("I failed 47 times before...")

Rule: aisa likho jaise user apne dost ko bata raha ho ki kya hua. Wahi tone.
```

SKILLS 9-14

Coding

Commit se lekar refactor tak. Sab coding tools ke liye.

09 CODING
commit-writer

YE KYA KARTA HAI

Tere code changes dekh ke saaf commit message likhta hai. 'fix bug' type message band.

KAB USE KARE

Har commit se pehle.

BOL KE DEKH

`commit kar de`

Copy this into `~/.claude/skills/commit-writer/SKILL.md`

```
---
name: commit-writer
description: Write a clear commit message from the staged diff. Use when the user is
about to commit, asks for a commit message, or says "commit kar de".
---

# Commit Writer

Pehle `git diff --staged` chalao. **Diff padhe bina message mat likho.**

Format:
```
<type>: <ek line, 72 char se kam, present tense>

<kyun badla, agar obvious na ho>
```

Type: `feat`, `fix`, `refactor`, `docs`, `test`, `chore`, `perf`

Rules:
1. **Subject line mein bolo ki kya badla, kyun nahi.** Kyun body mein.
2. Present tense: "add login check", "added" nahi.
3. Agar diff mein 3 se zyada alag-alag cheezein hain, bol do ki isko alag commits mein
todna chahiye. Sab ek message mein mat thoso.
4. Body tabhi likho jab "kyun" code se saaf na ho. Warna chhod do.
5. Bug fix hai to likho ki bug tha kya, sirf fix nahi.

Ye kabhi mat likho: "update files", "fix bug", "changes", "wip"
```

10 CODING

test-writer

YE KYA KARTA HAI

Kisi bhi file ke liye tests likhta hai. Wo cases jo sach mein tootte hain: empty input, null, galat type.

KAB USE KARE

Naya code likha ho aur test zero ho.

BOL KE DEKH

`use test-writer on utils/parser.py`

Copy this into `~/.claude/skills/test-writer/SKILL.md`

```
---
name: test-writer
description: Write tests for a file or function, covering the cases that actually break.
Use when the user asks for tests, or ships code with no test coverage.
---

# Test Writer

Pehle file padho aur project ka existing test setup dekho. **Wahi framework aur wahi
style use karo** jo pehle se hai. Naya framework mat le aao.

Har function ke liye ye cases likho:
1. **Happy path:** normal input, normal output
2. **Edge:** empty, zero, ek item, bahut bade input
3. **Bad input:** null, galat type, missing field
4. **Error:** jo fail hona chahiye, wo fail ho raha hai ya nahi

Test ka naam bataye ki kya test ho raha hai:
`test_returns_empty_list_when_no_matches`, `test_1` nahi.

Rules:
- Ek test ek cheez check kare.
- Implementation ke internals test mat karo. Behaviour test karo.
- Agar test likhte waqt koi asli bug dikhe, **ruk ke bata do**. Test likh ke bug ko
chhupao mat.
- Test mein random ya current time use mat karo. Har baar same result aana chahiye.
```

11

CODING

code-explainer

YE KYA KARTA HAI

Anjaan code ko samjhata hai: ye karta kya hai, data kahan se kahan jaata hai, aur kahan haath lagane se kuch tootega.

KAB USE KARE

Naye codebase mein ghuse ho, ya kisi ka code samajh nahi aa raha.

BOL KE DEKH

`use code-explainer on src/auth/middleware.ts`

Copy this into `~/.claude/skills/code-explainer/SKILL.md`

```
---
name: code-explainer
description: Explain unfamiliar code: what it does, why it is built that way, and where
the tricky parts are. Use when the user pastes code and asks what it does or opens an
unfamiliar repo.
---

# Code Explainer

Upar se neeche, isi order mein:

1. **Ek line:** ye code karta kya hai.
2. **Data ka safar:** input kya aata hai, kya hota hai, output kya jaata hai.
3. **Line by line** sirf un hisson ka jo confusing hain. Har line explain mat karo. `i =
0` ko samjhane ki zaroorat nahi.
4. **Ye aisa kyun likha hai:** agar koi ajeeb design dikhe, usne aisa kyun kiya hoga.
Kabhi-kabhi wajah historical hoti hai.
5. **Dhyan dene wali jagah:** yahan bug aa sakta hai, ya yahan haath lagaya to kuch aur
tootega.

Rules:
- Agar code galat lag raha hai, bol do. Sirf describe mat karo.
- Jargon use karo par pehli baar define karke.
- Agar bina baaki repo dekhe samajh na aaye, bol do ki kaunsi file chahiye. Guess mat
karo.
```

12 CODING

bug-repro

YE KYA KARTA HAI

Bug ko pehle pakadta hai (ek failing test banata hai), phir fix karta hai. Andhere mein teer nahi.

KAB USE KARE

Jab bhi kuch toota ho. Fix se pehle, hamesha.

BOL KE DEKH

`use bug-repro: login fails when email has capital letters`

Copy this into `~/.claude/skills/bug-repro/SKILL.md`

```
---
name: bug-repro
description: Turn a bug report into a minimal reproduction and a failing test before any
fix is attempted. Use when the user reports a bug, pastes an error, or says something is
broken.
---

# Bug Repro

**Fix likhne se pehle bug ko pakdo.** Bina reproduce kiye fix karna sirf guess hai.

Steps:
1. **Report se nikalo:** kya hona chahiye tha, kya hua, kaise hua. Agar teeno saaf nahi
hain, poocho. Guess mat karo.
2. **Sabse chhota reproduction banao.** Extra code hata do jab tak bug bacha rahe.
3. **Ek failing test likho** jo is bug ko pakde. Chalao. **Confirm karo ki fail ho raha
hai**, aur sahi wajah se fail ho raha hai.
4. **Ab jaake root cause dhundo.** Symptom nahi, wajah.
5. Fix karo. Test pass hona chahiye. Baaki tests bhi chalao.

Rules:
- Agar reproduce nahi kar paa rahe, bol do. Jhootha fix mat do.
- Test bug ke behaviour ka naam le: `test_login_fails_when_email_has_uppercase`
- Fix ke baad poocho: "isi type ka bug aur kahan ho sakta hai?"
```

13

CODING

pr-reviewer

YE KYA KARTA HAI

Push karne se pehle tera diff review karta hai. Pehle asli bugs, baad mein safai.

KAB USE KARE

PR kholne se pehle.

BOL KE DEKH

`use pr-reviewer before I push`

Copy this into `~/.claude/skills/pr-reviewer/SKILL.md`

```
---
name: pr-reviewer
description: Review a diff before it is pushed, looking for correctness bugs first and
cleanup second. Use when the user is about to open a PR or asks for a review of their
changes.
---

# PR Reviewer

`git diff main...HEAD` chalao. Poora diff padho.

Isi priority mein dekho:

**1. Correctness (sabse pehle)**
- Aisa input jo crash kara de: null, empty, bahut bada
- Off-by-one, galat comparison
- Error handle nahi kiya
- Race condition, ya galat order

**2. Kuch toota to nahi**
- Koi function ka behaviour badla jise dusri jagah use karte hain?
- Database ya API ka shape badla?

**3. Safai (aakhir mein)**
- Duplicate code jo pehle se kahin hai
- Simple likha ja sakta tha
- Naam jo matlab nahi bata rahe

Har finding ke liye: **file:line, dikkat kya hai, aur ye kab fatega.**

Rules:
- Style pe mat atko agar linter already hai.
- Agar sach mein kuch galat nahi hai, bol do "theek hai". Bekaar ke comments mat banao
sirf dikhane ke liye.
- Jo bade issue hain wo upar. Nitpick neeche.
```

14 CODING

refactor-planner

YE KYA KARTA HAI

Code saaf karne se pehle plan banata hai: kya badlega, kis order mein, aur kaise pata chalega ki kuch toota nahi.

KAB USE KARE

Jab kuch bada restructure karne ka man ho.

BOL KE DEKH

`use refactor-planner for the payments module`

Copy this into `~/.claude/skills/refactor-planner/SKILL.md`

```
---
name: refactor-planner
description: Plan a refactor before touching any code: what changes, in what order, and
how to verify nothing broke. Use when the user wants to clean up, restructure, or rewrite
part of a codebase.
---

# Refactor Planner

**Pehla sawaal: refactor karna zaroori bhi hai?** Agar code kaam kar raha hai aur tu use
chhune wala nahi hai, chhod de. Bol de ki chhod de.

Agar zaroori hai:
1. **Abhi kya hai:** ye code karta kya hai, aur kaun-kaun ise use karta hai. Callers
dhundo. Poore repo mein.
2. **Dikkat kya hai:** asli dikkat likho. "purana lag raha hai" dikkat nahi hai.
3. **Baad mein kya hoga:** target shape.
4. **Chhote steps mein todo.** Har step ke baad code chalna chahiye. Ek bada "sab badal
do" step nahi.
5. **Verify kaise karenge:** kaunse test, ya kya manually check karna hai.
6. **Risk:** kya toot sakta hai, aur wapas kaise jayenge.

Rule: agar tests nahi hain, **pehle tests likho, phir refactor.** Bina test ke refactor
sirf ummeed hai.
```

SKILLS 15-18

Data aur office

Spreadsheets, bills, meetings, inbox.

15 DATA AUR OFFICE
sheet-cleaner

YE KYA KARTA HAI

Messy CSV ya spreadsheet mein duplicates, galat formats, khaali cells dhundhta hai. Pehle report deta hai, phir tere bolne pe saaf karta hai.

KAB USE KARE

Jab koi export mila ho jo bikhra hua hai.

BOL KE DEKH

`use sheet-cleaner on leads.csv`

Copy this into `~/.claude/skills/sheet-cleaner/SKILL.md`

```
---
name: sheet-cleaner
description: Clean a messy spreadsheet or CSV (duplicates, inconsistent formats, missing
values) and report every change. Use when the user shares a sheet, CSV, or exported data
that needs tidying.
---

# Sheet Cleaner

**Pehle dekho, phir badlo.** Kuch bhi change karne se pehle report do ki kya-kya dikkat
hai. User ko decide karne do.

Kya check karna hai:
1. **Duplicate rows:** poore same, aur "lagbhag same" (spacing ya case ka farak)
2. **Format alag-alag:** dates (DD/MM vs MM/DD), phone numbers, currency
3. **Khaali cells:** kitne, kaunse column mein
4. **Type galat:** number wale column mein text
5. **Outliers:** jo values galti lag rahi hain

Phir report do:
```
Column   Rows   Khaali   Dikkat
email   1,240   12   31 duplicate, 4 mein @ nahi
joined_date   1,240   0   do format mile: DD/MM/YY aur YYYY-MM-DD
```

Rules:
- **Original file kabhi mat badlo.** Nayi copy banao.
- Har change ka log rakho. "1,240 mein se 31 rows hatayi". Chupke se mat karo.
- Agar samajh na aaye ki koi row duplicate hai ya nahi, **pooch lo**. Data delete karna
wapas nahi hota.
```

16

DATA AUR OFFICE

invoice-parser

YE KYA KARTA HAI

Bills aur receipts se line items nikaal ke table banata hai, GST alag, total neeche.

KAB USE KARE

Kharche jodne ho, ya reimbursement file karni ho.

BOL KE DEKH

`use invoice-parser on these 4 receipts`

Copy this into `~/.claude/skills/invoice-parser/SKILL.md`

````
---
name: invoice-parser
description: Pull structured line items out of invoices, bills, or receipts and total
them up. Use when the user shares a bill, invoice, or receipt image or PDF and wants the
data.
---

# Invoice Parser

Har invoice se nikalo:
- Vendor ka naam
- Invoice number aur date
- Har line item: description, quantity, rate, amount
- Subtotal, tax (GST alag se), total
- Currency

Table mein do:
```
Date         Vendor      Item               Amount
2026-08-04   Blue Dart   Courier - Mumbai   Rs 420.00
2026-08-11   AWS         EC2 - July         Rs 8,240.00
                         Total              Rs 8,660.00
```

Rules:
- **Numbers exactly waise hi likho jaise likhe hain.** Round mat karo.
- Agar line items ka jod total se match na kare, **bol do**. Chhupao mat. Ya to tax hai
ya bill mein galti hai.
- Agar koi field dhundhla ya kata hua hai, `[unclear]` likho. Guess mat karo.
- Multiple currency ho to alag rakho, khud convert mat karo.
````

17

DATA AUR OFFICE

meeting-notes

YE KYA KARTA HAI

Meeting transcript se decisions, action items aur open questions nikaalta hai. Poori baat-cheet nahi, sirf kaam ki cheez.

KAB USE KARE

Har meeting ke baad.

BOL KE DEKH

`use meeting-notes on transcript.txt`

Copy this into `~/.claude/skills/meeting-notes/SKILL.md`

````
---
name: meeting-notes
description: Turn a meeting recording or transcript into decisions, action items, and
open questions. Use when the user shares a call recording, transcript, or meeting notes.
---

# Meeting Notes

Output isi order mein. Sabse kaam ki cheez upar:

**1. Decisions:** kya tay hua. Har ek ke saath kisne tay kiya.

**2. Action items:** sabse important hissa.
```
[ ] <kaam> | <kaun> | <kab tak>
```
Agar kisi ka naam ya date nahi bola gaya, likho `[owner nahi mila]`. **Kisi ka naam apne
aap mat daalo.**

**3. Open questions:** jo cheezein latki rahin.

**4. Baaki baatein:** context jo baad mein kaam aa sakta hai. Chhota rakho.

Rules:
- **Poori baat-cheet mat likho.** Koi transcript nahi padhta.
- Small talk, "hello hi", technical dikkat: sab hata do.
- Agar kisi cheez pe jhagda hua aur solve nahi hua, likho ki solve nahi hua.
- Agar audio saaf nahi tha aur kuch samajh nahi aaya, `[unclear]` likho.
````

18

DATA AUR OFFICE

email-triage

YE KYA KARTA HAI

40 emails ko 4 buckets mein baant deta hai aur jahan reply chahiye wahan draft likh deta hai. Bhejta kabhi nahi, tu bhejega.

KAB USE KARE

Subah, ya jab inbox dekhne ka dar lag raha ho.

BOL KE DEKH

`use email-triage, here are today's emails: (paste)`

Copy this into `~/.claude/skills/email-triage/SKILL.md`

```
---
name: email-triage
description: Sort a batch of emails by what actually needs action and draft the replies.
Use when the user pastes emails or asks for help clearing their inbox.
---

# Email Triage

Har email ko char mein se ek bucket do:
1. **Aaj reply karna hai:** koi wait kar raha hai, ya deadline hai
2. **Reply karna hai, jaldi nahi:** is hafte
3. **Padhna hai, reply nahi:** info hai bas
4. **Kuch nahi karna:** newsletter, notification, spam

Bucket 1 aur 2 ke liye **draft reply likho.**

Reply ka style:
- Chhota. 3-5 lines. Lamba email koi nahi padhta.
- Pehli line mein kaam ki baat. "Hope you are doing well" nahi.
- Agar koi commitment de rahe ho to date daalo.
- Sender ka tone match karo. Formal ko formal, casual ko casual.

Rules:
- **Kabhi khud se bhejo mat.** Draft do, user bhejega.
- Agar reply ke liye koi info chahiye jo user hi de sakta hai, blank chhod do: `[yahan
date daalo]`
- Agar koi email dhang ka nahi lag raha (phishing type), bol do.
```

SKILLS 19-22

Content banane wale

Reels, thumbnails, captions, repurposing.

19 CONTENT BANANE WALE
reel-scripter

YE KYA KARTA HAI

Ek idea se 30 second ka Hinglish reel script, har line ke saamne b-roll idea ke saath.

KAB USE KARE

Roz post karna ho aur script likhne mein 1 ghanta jaa raha ho.

BOL KE DEKH

`use reel-scripter: why most people quit gym in February`

Copy this into `~/.claude/skills/reel-scripter/SKILL.md`

```
---
name: reel-scripter
description: Turn an idea into a short vertical video script in Roman Hinglish, built for
retention. Use when the user wants a reel script, short script, or says "iska reel banana
hai".
---

# Reel Scripter

Length: **30 second se kam. 75-85 words.** Isse zyada mein log nikal jaate hain.

Structure:
1. **Hook (0-3 sec):** ek specific line. Yahi decide karta hai ki reel chalega ya nahi.
Number, ya ek surprising baat.
2. **Tension (3-8 sec):** problem. Wo cheez jo dekhne wale ko chubhti hai.
3. **Payload (8-25 sec):** asli value. Ek cheez, saaf tarike se.
4. **Landing (25-30 sec):** ek line jo yaad rah jaye.

Rules:
- **Roman Hinglish. Devanagari nahi.**
- Sentences chhote. 10 word se kam.
- Aise likho jaise bol rahe ho, likh nahi rahe. Zor se padh ke dekho.
- Overclaim mat karo. "insane", "game changer", "life-changing": nahi.
- Ek reel = ek idea. Do idea = do reel.

Script ke saath do: har line ke saamne **screen pe kya dikhega** (b-roll idea).
```

20

CONTENT BANANE WALE

thumbnail-brief

YE KYA KARTA HAI

Video ke liye 3 alag thumbnail concepts: text, visual, expression, aur click kyun hoga.

KAB USE KARE

Upload se pehle.

BOL KE DEKH

`use thumbnail-brief for my video on Claude Code skills`

Copy this into `~/.claude/skills/thumbnail-brief/SKILL.md`

```
---
name: thumbnail-brief
description: Generate thumbnail concepts for a video, with the text, the visual, and why
each would get a click. Use when the user is about to publish a video and needs a
thumbnail idea.
---

# Thumbnail Brief

**3 alag concepts** do. Alag angle se, ek hi cheez ke 3 version nahi.

Har concept ke liye:
- **Text on screen:** **4 word se kam.** Phone pe thumbnail chhota dikhta hai.
- **Visual:** frame mein kya hai. Face? Object? Before/after?
- **Expression:** agar chehra hai to kaisa
- **Click kyun karega:** ek line mein wajah

Jo kaam karta hai:
- Ek cheez pe focus, bhari hui image nahi
- High contrast, kyunki feed mein chhota dikhega
- Face ho to aankhein camera mein
- Text aur background ka rang alag

Jo kaam nahi karta:
- Poora sentence
- 3 se zyada element
- Chhote details jo phone pe dikhenge hi nahi
- Aisa thumbnail jo video ke content se match na kare. Log turant nikal jaate hain.

Rule: title aur thumbnail ek dusre ko repeat na karein. Dono milke kahani banayein.
```

21

CONTENT BANANE WALE

caption-writer

YE KYA KARTA HAI

Platform ke hisaab se caption aur hashtags. Instagram, LinkedIn, X, YouTube sab alag chalte hain, ye jaanta hai.

KAB USE KARE

Post publish karne se theek pehle.

BOL KE DEKH

`use caption-writer for Instagram, video is about 5 AI tools for students`

Copy this into `~/.claude/skills/caption-writer/SKILL.md`

```
---
name: caption-writer
description: Write platform-appropriate captions and hashtags for a post. Use when the
user has a video, reel, or image ready and needs the caption.
---

# Caption Writer

Poocho ki platform kaunsa hai: Instagram, LinkedIn, X, YouTube. Har jagah alag chalta
hai.

**Instagram:**
- Pehli line hook. Baaki "more" ke peeche chhupega
- 100-150 words
- 5-8 hashtags, mix: bade aur chhote dono
- Ek CTA, wo bhi tabhi jab sach mein chahiye

**LinkedIn:**
- Pehli 2 lines sab kuch hain
- 150-200 words, chhote paragraph
- 3-5 hashtags, aakhir mein
- Story pehle, lesson baad mein

**X:**
- 280 char, ya thread
- Thread ho to har tweet apne aap mein poora ho
- Hashtag mat use karo, X pe kaam nahi karte

**YouTube:**
- Pehli 2 lines search mein dikhti hain. Keyword wahin daalo
- Timestamps agar video 3 min se lamba hai

Rule: 3 options do, alag-alag tone mein. User chunega.
```

22

CONTENT BANANE WALE

content-repurposer

YE KYA KARTA HAI

Ek lambi video ya blog se 5-8 chhote pieces nikaalta hai: reel, carousel, thread, post. Har ek platform ke liye dobara likha hua.

KAB USE KARE

Jab ek accha piece bana ho aur usse zyada nikalna ho.

BOL KE DEKH

`use content-repurposer on my 12 min YouTube video transcript`

Copy this into `~/.claude/skills/content-repurposer/SKILL.md`

```
---
name: content-repurposer
description: Break one long piece of content into many smaller platform-specific pieces.
Use when the user has a video, blog post, or long thread and wants more out of it.
---

# Content Repurposer

Source padho/dekho. **Har wo point nikalo jo apne aap mein khada ho sakta hai.**

Har point ke liye batao ye kahan chalega:

| Format        | Kitna lamba | Kis type ka point    |
|---------------|-------------|----------------------|
| Reel / Short  | 30 sec      | Ek surprising baat   |
| Carousel      | 5-8 slide   | Steps ya list        |
| Single post   | 150 words   | Ek opinion           |
| Thread        | 5-10 tweet  | Ek argument          |
| Newsletter    | 400 words   | Poora context        |

Rules:
- **Sirf format mat badlo.** Har platform ke liye dobara likho. Blog ka paragraph reel
mein paste karna bekaar hai.
- Ek source se 5-8 cheezein nikalni chahiye. Isse kam matlab source patla tha.
- Sabse strong point ko sabse bade platform pe do.
- Batao ki kis order mein aur kitne gap mein post karna hai.
```

SKILLS 23-25

Roz ka kaam

Review, decisions, trips.

23 ROZ KA KAAM
weekly-review

YE KYA KARTA HAI

Hafte ke notes, commits, messages dekh ke honest review: kya hua, kya chala, kya nahi, aur agle hafte ki sirf 3 cheezein.

KAB USE KARE

Sunday shaam.

BOL KE DEKH

`use weekly-review, here is my week: (paste notes)`

Copy this into `~/.claude/skills/weekly-review/SKILL.md`

```
---
name: weekly-review
description: Turn a week of notes, commits, or messages into an honest review and a plan
for next week. Use when the user wants a weekly review or retrospective.
---

# Weekly Review

Jo mila hai wo padho: notes, commits, tasks, messages.

**1. Kya hua:** sirf facts. Jo sach mein complete hua. Jo shuru karke chhod diya wo alag
likho.

**2. Kya chala:** kaunsi cheez ne sach mein farak daala. Har kaam nahi. Wo do teen jo
mattered.

**3. Kya nahi chala:** honestly. Kya atka, kyun atka.
- Agar ek hi cheez teesre hafte se pending hai, **bol do.** Wo shayad hoga hi nahi, ya
use todna padega.

**4. Agle hafte:** **3 cheezein. Bas 3.** Priority ke order mein. Har ek ke saath: pehla
step kya hai.

Rules:
- Motivational baatein mat karo. Ye review hai, pep talk nahi.
- Agar hafta kharab gaya, likho ki kharab gaya.
- Agar user ne apni capacity se zyada plan kiya hai, bol do.
```

24 ROZ KA KAAM

decision-log

YE KYA KARTA HAI

Koi bhi decision, uske options aur wajah ko ek page mein likh ke rakhta hai. 3 mahine baad 'ye kyun kiya tha?' ka jawab yahin milega.

KAB USE KARE

Jab koi technical ya business decision liya ho jo yaad rakhna zaroori hai.

BOL KE DEKH

`use decision-log: we chose Supabase over Firebase`

Copy this into `~/.claude/skills/decision-log/SKILL.md`

````
---
name: decision-log
description: Capture a decision, the options considered, and the reasoning, in a format
that will still make sense months later. Use when the user makes a technical or business
decision worth remembering.
---

# Decision Log

Format:
```
# <decision, ek line mein>
Date: <YYYY-MM-DD>

## Kya tay kiya
<ek paragraph>

## Kyun
<asli wajah, jo sach mein decide karwaya>

## Aur kya options the
- <option>: chhoda kyunki <wajah>
- <option>: chhoda kyunki <wajah>

## Kya maan ke chal rahe hain
<wo cheezein jo agar galat nikli to ye decision galat ho jayega>

## Kab dobara sochenge
<date ya trigger>
```

Rules:
- **Reject kiye hue options zaroor likho.** 6 mahine baad wahi sabse kaam ka hissa hota
hai. Warna wahi bahas dobara hoti hai.
- "Assumptions" wala hissa mat chhodo. Wahi batata hai ki decision ab bhi sahi hai ya
nahi.
- Chhota rakho. Ek page. Lamba likhoge to kabhi likhoge hi nahi.
````

25 ROZ KA KAAM
trip-planner

YE KYA KARTA HAI

Budget, dates aur pasand se asli itinerary. Din mein max 3 cheezein, travel time mein traffic jod ke, kharcha alag-alag.

KAB USE KARE

Ghumne ka plan bane.

BOL KE DEKH

`use trip-planner: Udaipur, 3 days, 2 people, 15k budget`

Copy this into `~/.claude/skills/trip-planner/SKILL.md`

````
---
name: trip-planner
description: Build a realistic trip itinerary from a budget, dates, and interests, with
honest travel times. Use when the user is planning a trip and needs a day-by-day plan.
---

# Trip Planner

Chahiye: kahan, kitne din, budget, kitne log, kya pasand hai. Missing ho to poocho.

Har din ke liye:
```
Day 3: Udaipur
Morning     City Palace (Rs 300, 2-3 ghante)
Afternoon   Lunch + rest. Dopahar mein bahar mat nikalna
Evening     Lake Pichola sunset boat (Rs 400, 1 ghanta)
Kharcha     Rs 1,400 + khana
```

Rules jo plan ko asli banate hain:
1. **Ek din mein 3 se zyada cheezein mat daalo.** Har itinerary isi mein fail hoti hai.
2. **Travel time asli likho.** Google jo bata raha hai usme traffic add karo.
3. **Har din ek khaali slot chhodo.** Kuch na kuch late hoga.
4. **Kharcha alag-alag likho:** rehna, aana-jaana, khana, ticket. Total neeche.
5. Agar budget mein plan nahi ban raha, **bol do.** Kya kaatna padega batao.
6. Local timings ka dhyan: kya band rehta hai kis din, kya advance book karna hai.
````

AGLA LEVEL

Apni skill kaise banaye

Ab tak tune dekha ki har skill ka shape ek hi hai. Apni banana utna hi easy hai.

1. **Wo kaam dhundh jo tu baar-baar prompt karta hai**

   Hafte mein 3 baar same cheez samjha raha hai? Wo skill hai.

2. **Folder bana**

```
mkdir -p ~/.claude/skills/mera-kaam
```

3. **SKILL.md likh, isi template se**

```
---
name: mera-kaam
description: <kya karta hai>. Use when <kab trigger ho>.
---

# Mera Kaam

<steps, order ke saath>

Rules:
- <kya nahi karna>
```

4. **Test kar**

   Kaam bol, dekh skill uthti hai ya nahi. Nahi uthi to description sudhaar. 99% baar wahi problem hoti hai.

Achhi skill vs bekaar skill

Bekaar	Achhi
`description: makes notes`	`description: Turn a PDF into structured notes. Use when the user shares a PDF or asks for a summary.`
Body mein sirf "be helpful"	Body mein asli steps, order ke saath
Koi rule nahi	"Ye mat karna" wali list
10 kaam ek skill mein	Ek skill, ek kaam

Sabse bada sabak

Skill likhna matlab apni soch likhna. Tu jo kaam achhe se karta hai, uske steps likh de. Bas wahi skill hai.

BAS ITNA HI

25 skills.
Sab copy-paste ready.

Ek saath sab mat daal. Do se teen le, ek hafta chala, phir aur add kar. Jo skills tu use nahi karta wo sirf clutter hain.

Agar koi skill kaam na kare: pehle description dekh. Hamesha wahi hota hai.

Kuch bana? Bhej dena @ai_with_mudit pe. Dekhna accha lagega.